Check for updates

Dule Shu
Department of Mechanical Engineering,
Carnegie Mellon University,
Pittsburgh, PA 15213
e-mails: djs6106@psu.edu;
dules@andrew.cmu.edu

James Cunningham
Department of Mechanical Engineering,
Carnegie Mellon University,
Pittsburgh, PA 15213
e-mails: jdc5549@psu.edu;
jamescun@andrew.cmu.edu

Gary Stump
Applied Research Laboratory,
The Pennsylvania State University,
University Park, PA 16802
e-mails: gms158@psu.edu;
gms158@arl.psu.edu

Simon W. Miller
Applied Research Laboratory,
The Pennsylvania State University,
University Park, PA 16802
e-mail: swm154@arl.psu.edu

Michael A. Yukish
Applied Research Laboratory,
The Pennsylvania State University,
University Park, PA 16802
e-mail: may106@arl.psu.edu

Timothy W. Simpson
Mem. ASME
Department of Mechanical Engineering,
The Pennsylvania State University,
University Park, PA 16802
e-mail: tws8@psu.edu

Conrad S. Tucker
Mem. ASME
Department of Mechanical Engineering,
Carnegie Mellon University,
Pittsburgh, PA 15213
e-mail: conradt@andrew.cmu.edu

3D Design Using Generative Adversarial Networks and Physics-Based Validation

The authors present a generative adversarial network (GAN) model that demonstrates how to generate 3D models in their native format so that they can be either evaluated using complex simulation environments or realized using methods such as additive manufacturing. Once initially trained, the GAN can create additional training data itself by generating new designs, evaluating them in a physics-based virtual environment, and adding the high performing ones to the training set. A case study involving a GAN model that is initially trained on 4045 3D aircraft models is used for demonstration, where a training data set that has been updated with GAN-generated and evaluated designs results in enhanced model generation, in both the geometric feasibility and performance of the designs. Z-tests on the performance scores of the generated aircraft models indicate a statistically significant improvement in the functionality of the generated models after three iterations of the training-evaluation process. In the case study, a number of techniques are explored to structure the generate-evaluate process in order to balance the need to generate feasible designs with the need for innovative designs. [DOI: 10.1115/1.4045419]

Keywords: computer-aided design, conceptual design, design automation

1 Introduction

The emergence of generative design methods is accelerating the pace at which designers can explore and refine their ideas. Specifically, deep learning-based generative design tools and approaches provide designers with a scalable means of generating novel design concepts [1–3]. One major advantage of deep learning methods over other data-driven methods is the ability of the neural network models to learn the features of a design, with minimal input from the designer [4,5]. For instance, in many popular generative models, the input variable of a particular layer is often used as a lower-dimensional representation of the original design. Once a neural network is properly trained, it associates designs with a lower-dimensional representation, also known as the feature or latent variable space. To a human designer, the feature space is an alternative perspective to analyze a design problem. In addition to tuning and comparing different design concepts, the designer can tune and compare their corresponding features searching for better designs or for studying the underlying connections between designs. Since features are a more compressed description of the essential characteristics of design concepts, analyzing a design problem in the feature space can potentially extract key information which is implicitly contained in the original design space.

Deep learning-based generative models such as generative adversarial networks (GANs) [6,7] or recurrent neural networks [8,9] can be trained to discover features of a design underneath its visual appearance; however, in the context of concept generation for design, there is a significant amount of domain knowledge embedded in a designer's visual interpretation of a design that extends beyond the design's form. The feasibility of design is based on its *form* (i.e., its geometric properties), *function* (i.e., its intended purpose), and *behavior* (i.e., how well it achieves its intended

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received March 30, 2019; final manuscript received October 12, 2019; published online November 6, 2019. Assoc. Editor: Xiaoping Qian.

 Copyright © 2019 by ASME

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

purpose when interacting with an environment or entity). There has been a significant amount of work in the design community exploring the relationship between form, function, and behavior [10–13]. While it is relatively easy to train a human designer to make cognitive connections between form, function, and behavior when observing a visual representation of a design, it is much more challenging to train a computer to do the same. This challenge is at the core of a new and open research area.

Consider two neural network-generated 3D models of aircraft concepts as shown in Fig. 1. The generated designs in Fig. 1 would be evaluated based on several design conditions:

- Does this 3D model accurately capture the *form* of an aircraft? This question can be addressed by simply asking other members of a design team for feedback pertaining to a generated design solution, otherwise known as a design critique [14]. Designers draw upon their vast experience of visually observing previous designs and sketches and respond accordingly. In a similar manner, deep learning models can use a large repository of existing 3D aircraft models and other designs to determine whether the generated aircraft does indeed look like an aircraft, given a wide range of different classes to choose from [2].
- Does this 3D model meet its intended *function(s)*? In many cases, a design concept's ability to meet its intended function is correlated to its form, in addition to the functional constraints of the environment. In Fig. 1, if the primary objective of the aircraft is to achieve good aerodynamic performance, then a designer may conclude that the model on the left is superior to the model on the right. This domain knowledge of the human designer is based on a fundamental understanding of the laws of physics. This domain knowledge enables designers to make a connection between a generated design, the interaction of the generated design with its intended environment (e.g., air), and an understanding that a more streamlined fuselage with wider wings may generate more lift and less resistance in the air.
- Does this 3D model accurately capture the behavior of an aircraft? The ability of a designer to predict whether a design will achieve its intended behavior is a multifaceted problem that includes the selection of the material(s) from which the design is created, the environmental conditions, and so on. Considering aspects of the design outside the geometric form is out of the scope of this work.

Knowledge gap: Unlike human designers, existing generative models have up until now primarily learned the visual aspects of a design absent to how those visual aspects of the design relate to functional characteristics of its environment. This is because, in creating the training data for the generative models, the visual aspects of a design are primarily taken into consideration. As a result, little guarantee is made on the fidelity of the designs from the training data set in terms of their functional performance. The authors propose to bridge the knowledge gap between the visual aspects of design, and its corresponding functional performance, by updating the training models for the neural network with well-functioning generated models selected through a virtual physics-based simulation. While design schemas and data repositories for designs exist [15], they require the manual labeling of data by humans, hereby severely constraining the size and availability of training data for machine learning algorithms. The authors hypothesize that training data that include machine-validated designs from a physics-based virtual environment, increases the likelihood of generative models creating functionally feasible design concepts. Preliminary studies by the authors explored this problem in terms of 2D design sketches [16]. In this paper, the authors explore the complexity of 3D model generation and evaluation. Advancing from 2D generative design to 3D generative design creates new technical challenges, as the data representing 3D objects are more voluminous and complex than the data representing 2D shapes. To address these technical challenges, a new GAN design is proposed in this paper and evaluated based on its ability to generate both visually and functionally feasible designs. The scientific contributions and novelty of this work are as follows:

(1) A neural network method that combines 3D point-cloud generation and 3D mesh reconstruction for performance evaluation of engineering design concepts in their native format (i.e., a mesh). With the emergence of open-source repository such as ShapeNet, Thingiverse, and GrabCAD, data sets of engineered systems ranging from chairs to aircrafts can be used as training data to generate new designs with minimal engineering domain expertise required.
(2) Design space exploration via linear interpolation and extrapolation in the latent space of the generative neural network model. This method enables morphing (transforming from Design A to Design B) and synthesizing (combining multiple designs such as an aircraft + car + boat) of different designs with minimal constraints.
(3) A method that enhances the fidelity of generated designs by iteratively updating the training data set using performance filtering. This iterative process has the potential to result in a statistically significant performance improvement of the generated designs.

The proposed performance filtering approach (as further described in Secs. 3.3 and 4.3) is chosen over other approaches such as pre-filtering of the initial training data, due to the following potential scenarios that could otherwise result; in the first scenario, the initial training data set has a limited size and a small portion of high-performance designs. If prefiltering of the training data set is performed, the remaining designs may not form a sufficiently large data set to ensure the performance of neural network training. As pointed out in Refs. [17,18], generative neural networks typically require a sufficiently deep model to disentangle the underlying factors of variation in the data distribution and enable diversity in generated samples. This, in turn, translates to the requirement of a large amount of training data. In the second scenario where the size of the training data set is sufficiently large, evaluating the performance of all designs in the data set may become a computationally expensive task. The two scenarios and the corresponding challenges to GAN training have been listed in Table 1.

This paper is organized as follows: This section provides an introduction to generative models for design and the challenges of

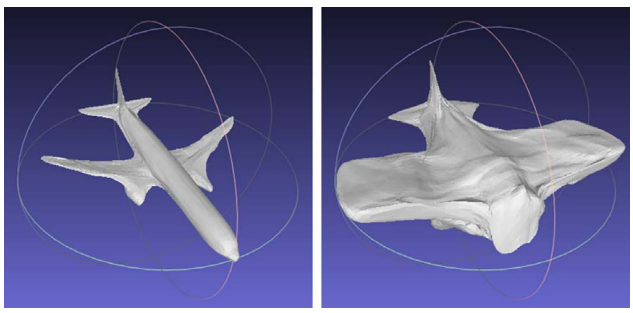


Fig. 1 Two neural network-generated 3D aircraft models

Table 1 Improper sizes of training data set and the corresponding challenges to GAN training

Size of the initial training data set	Potential challenge of pre-filtering on GAN training
Too small	Insufficient number of remaining data points for training
Too large	High computational cost for evaluating the performance of all training data points

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

embedding function and form into the training data used to train these models. Section 2 reviews literature most closely related to this work. Section 3 introduces a deep generative design model and presents a physics-based virtual simulation environment approach that can evaluate the functionality of generated design solutions. Section 4 introduces the case study that tests the hypothesis of this work, and Sec. 5 presents the results from the case study. Finally, Sec. 6 concludes the paper and discusses possible areas of future research expansion.

2 Literature Review

2.1 Automated Generative Design Methods. The automated design of 3D objects has been actively investigated in multiple aspects by the research community. Ulu and Kara [19] propose a method that automatically generates geometries from existing objects which improves the efficiency of shape customization. Andrade et al. [20] use cladding of panels and honeycomb structures to create patterns on top of base facades. The patterns are generated by calculating the barycenter coordinates of the simplicial in the mesh and generating a list of neighbors for each triangle. In developing shape grammar for designs, Whiting et al. [21] introduce a shape grammar for motorcycle that captures the brand identity by decomposing the brand into forms and their interrelations identified with functional features. Another class of approaches for automatic design generation, namely the deep generative designs, uses generative neural networks to generate new designs. The new designs are defined by 3D geometries of an object generated by a neural network. The 3D geometry can be represented in different ways. Some popular representations are point cloud, mesh, and voxels. A mesh representation approximates the 3D geometry of an object by stitching a group of polygons together. It is widely used in many software packages working with 3D models, such as UNITY, SOLIDWORKS, and OPENFOAM. A point-cloud representation can be considered as a simplification of a mesh where only the vertices of the polygons are used to describe the geometry. The voxel representation approximates an object by cells from a partitioned 3D space.

Generative adversarial networks (GANs) [6] combine a generator neural network with a discriminator neural network. The generator neural network is trained to learn the probability distribution of the training data so that it can generate new data samples from the same probability distribution. On the one hand, since the generated data samples are intended to satisfy the same probability distribution that the training data samples satisfy, they retain a certain level of similarity with the training data samples. On the other hand, since the similarity is defined in the sense of probability distribution, a generated data sample is highly unlikely to be identical to any particular data sample in the training data set. The difference between the training data and the generated data motivates the usage of GANs for design concept generation in this paper. In the context of GANs, data from the training data set are often called the "real data" as opposed to the data from the generator output which are called the "fake data." Given any data as input (either real or fake), the discriminator neural network is trained to compute a loss function whose value indicates how likely it is that the given input data are drawn from the same distribution as its training data. Based on the value of the loss function, the discriminator tries to differentiate the fake data from the real data. The performances of the generator and the discriminator are evaluated by the ground truth of the input training data (fake or real) during GAN training.

Variational autoencoders (VAEs) [22] are another generative neural network model that consists of two networks, namely, an encoder and a decoder. The *encoder* takes a batch of data samples as input, and outputs a lower-dimensional representation of the data named the latent variable. The latent variable is constructed using a random sample of a distribution with mean and variance computed by the encoder. The *decoder* takes the latent variable as input, and outputs variables of the same dimension as the original data. A typical VAE model is trained to generate a close reconstruction of its input, while enabling the latent variable to be a compact representation of the input. From a theoretical perspective, both GANs and VAEs are approaches that estimate a probability distribution. Genevay et al. [23] suggest that the formulation of these two generative models can be related to the same minimum Kantorovitch estimation problem. Unlike VAEs that formulate the estimated distribution as a marginal likelihood conditioned on some latent variable z [24], the generator of GANs directly takes samples from distribution as input. Thus, the generator is only updated with gradient flowing through the discriminator, which makes the generator more independent of the component of training data [6]. Not involving conditional probability during data generation allows GANs to represent sharp distributions, while estimators relying on conditional probability, such as VAEs, require the distribution be "somewhat blurry" [6] so that the probability chains are able to mix between modes. Hence, although the GAN model tends to suffer from instability during training, the authors decide to choose it over the more stable VAE, in order to introduce more variability in the generated designs. Mode collapse is a potential problem in the traditional GAN model used in this paper. As pointed out by Arjovsky and Bottou [25], the problem of mode collapse becomes significant when the discriminator is trained to become optimal or near-optimal. Hence, one way of reducing the chance of encountering mode collapse is to restrain the training of discriminator such that the discriminator does not converge to optimality too quickly. In the proposed GAN model, the discriminator has a more complex structure than the generator, and both the generator and the discriminator are trained once per training iteration. This choice of discriminator model and GAN training has helped to set the training of the generator and the discriminator at a proper pace, and thus has reduced the possibility of mode collapse.

One potential advantage of the neural network-based generative design is that it provides an alternative way of tuning and analyzing the design concepts. For example, in addition to directly modifying the geometry of an existing design, one can modify the value of the corresponding latent variable (also named the feature) of that design. Moreover, the generative neural network, if properly trained, can form a continuous mapping from the latent space to the design space. As a result, a smooth change of value in the feature yields a smooth change of geometry in the design concept. To illustrate this, Fig. 2 shows an example process of model interpolation and

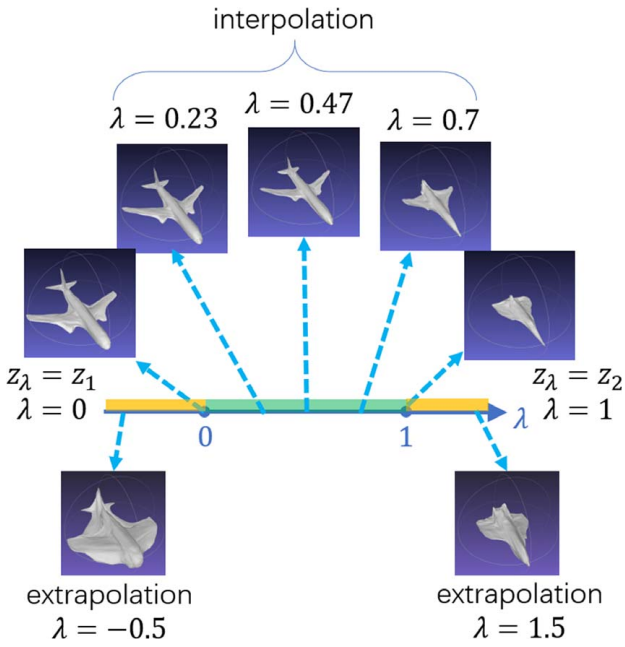


Fig. 2 Continuous transformation of the generated designs via linear combination of two latent variables

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

extrapolation implemented in the latent space. Define the linear combination of two latent variable samples as

$$\tilde{z} = (1 - \lambda)z_1 + \lambda z_2 \tag{1}$$

where

- $z_1, z_2 \in \mathbb{R}^n$ are two n-dimensional vectors defined as latent variable samples, and
- $\lambda \in \mathbb{R}$ is the controlling factor.

By varying the value of λ, the geometry of the generated model corresponding to $\tilde{z}$ can be varied. Figure 2 shows an example of design variation through latent-space interpolation ($0 \le \lambda \le 1$) and extrapolation ($\lambda = -0.5, 1.5$) using the GAN model presented in Sec. 3. As shown in Fig. 2, continuity can be observed in the change of generated model geometry as λ changes monotonically. Furthermore, the knowledge about the topology of the latent space can potentially be used for design optimization and design classification.

Generative models have been used to explore design spaces for product forms such as cars [1] and, to more broadly, define design spaces to be sampled [26,27]. These models have even been employed to guide physics-based approaches such as material microstructures [28,29]. For 3D data generation, in particular, various GAN models have been used to generate data in different forms. Wu et al. [30] present a GAN model for volumetric data generation, where 3D objects defined in a $64 \times 64 \times 64$ voxel space are generated from 200-dimensional latent vectors. The main challenge for voxel-based approaches is the high computational cost for data representation and the brick-like shape approximation for smooth surface objects. Compared with voxel models, mesh models require less computational cost for data representation and are more suitable for approximating smooth surfaces. A challenge of mesh generation is that the mesh model of a 3D surface is in general, defined in a non-Euclidian space, which makes it difficult to unify the data dimension and directly apply convolution operations to a continuous region on the surface. To address this challenge, Ben-Hamu et al. [31] propose a method that converts the 3D mesh representation of a surface to an image-like representation defined in a Euclidean 2D space using conformal toric charts. A GAN model is introduced to generate the 2D representations of the surface. The generated 2D representations are converted back to the 3D mesh space by solving the required scaling and translation parameters to shape a predefined template mesh. One limitation of this approach is its requirement for specifying a group of landmark vertices on the 3D surface. Tan et al. [32] propose a method that maps existing 3D mesh data from the surface mesh space to feature space and train a VAE to generate new features. This feature representation proposed by Gao et al. [33] consists of a rotation difference matrix of the edges and the scaling/shear matrix of the vertices. The generated features are eventually mapped back to the mesh space using the surface reconstruction method from Ref. [34]. However, the method assumes one-to-one correspondence of vertices between every two training models, which may not be satisfied by a training data set in general. As another popular way of representing 3D objects, the point-cloud model has higher computational efficiency compared with the mesh model, as it only contains the surface vertex coordinates and does not include any information about the edges or the faces. By fixing the number of vertices or points in a point cloud, the point-cloud data can be represented by a 2D matrix of fixed dimension. This data representation has been used by many generative network models such as autoencoder [34], VAE [35], and GANs [36,37]. In Ref. [36], three methods are proposed for point-cloud generation: the raw point-cloud GAN, the latent-space GAN, and the Gaussian mixture model approach. The latter two methods require that some latent representation of the point-cloud model be effectively learned, while the first method sometimes yields pathological cases in data generation due to the use of Chamfer distance in network training, as reported by its authors. Similar to Ref. [36], a latent representation of the point-cloud data needs to be inferred prior to data generation in the method proposed by Li et al. [37]. Note that the point cloud only contains partial information of the shape surface. In many shape generation problems, the complete shape surface is expected. In this case, the construction of a mesh model from a point-cloud model is required. One approach for mesh construction is to use a predefined baseline mesh model to provide face information of the point cloud. Another approach for mesh construction, as proposed in Ref. [35], is to incorporate surface point normal vectors into the data definition, so that mesh construction algorithms using surface normal vectors (e.g., Poisson surface reconstruction) can be applied. Such mesh construction methods cannot be used when normal vectors are not defined in the training data set. Although the values of the normal vectors can be estimated from the 3D coordinates of a point-cloud model, the performance of such estimation is dependent on the shape and density of a point cloud and is observed to be less robust than the mesh construction method employed in this paper. The GAN model in this paper is designed for point-cloud generation and does not require specific learning of latent or feature representations of a point cloud. The loss function of the GAN model is chosen as the classic loss function proposed in Ref. [6] and is presented as follows:

$$\min_G \max_D \; \mathcal{E}_{\boldsymbol{M_r} \sim p_{\boldsymbol{M_r}}}[\log(D(\boldsymbol{M_r}))] + \mathcal{E}_{z \sim p_z}[\log(1 - D(G(\boldsymbol{z})))] \tag{2}$$

where

- $\mathcal{E}$ denotes the expectation;
- G denotes the mathematical representation of the generator;
- D denotes the mathematical representation of the discriminator; and
- z is an m-dimensional random vector named the latent variable with a probability distribution chosen by the user (e.g., normal random distribution).

For mesh construction from generated point cloud, the approach in Ref. [34] is used and will be further described in Sec. 3 of this paper.

The method presented in this work advances the field of generative design by constraining the aggregation of training data that a simulation environment has evaluated and approximates the functional properties of the corresponding real-world environment. As a result, the authors postulate that the generated design solutions will be a mathematical function evaluation not only of a design's form (i.e., learned from point-cloud features) but also of its functionality (i.e., learned from physics-based simulations of real-world conditions).

2.2 Automated Design Evaluation Methods. The evaluation of design concepts is typically partitioned into form, function, and behavior evaluation [38,39]. Form evaluation focuses more on a design's ability to meet its intended esthetic objectives [40]. Evaluation of function and behavior focuses on a design artifact's ability to satisfy its performance objectives. Complex analyses tools such as finite element analysis (FEA) models along with computational fluid dynamics (CFD) have been extensively used in the design and engineering fields to model structural performance and fluid flows and interactions on a design concept [41–45]. Simulation tools such as the CFD visualization provide both a *visual* and a *functional* analyses of the performance/feasibility of a given design solution. Furthermore, sensitivity analysis can be performed wherein the functional characteristics of the simulation environment are varied in order to quantify the effects on a design concept [46]. Other well-known computational tools such as OPENVSP, MATLAB, or SOLIDWORKS have expanded their capabilities to give designers more tools to not only generate design solutions but also virtually evaluate said designs [47,48].

While automatic design evaluation tools exist, they are typically extremely computationally expensive. For example, Turrell [49]

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

reports that their CFD simulation of flow in a gas turbine combustor took several days to run. Furthermore, many of the well-established engineering simulation tools require deep expertise in software programming, optimization, and visualization [50]. The advancements in computing allow designers to generate more complex systems and higher fidelity analyses.

Topology optimization (TO) is an active research area for design optimization and design automation. Starting from an initial design, TO explores the design space to search for the ideal material distribution of a design that optimizes some user-specified performance index (i.e., stiffness and drag force in fluid). Areas of application for TO include solid mechanics [51–54], fluid dynamics [55–58], and additive manufacturing [59–62]. Among the various approaches used in TO, the solid isotropic material with penalization (SIMP) [63], the level set approach [64], and the evolutionary structural optimization [65] are three mainstream approaches in the research area, as summarized by Liu and Ma [66]. In SIMP, an equation to describe the relationship between the continuous density variable and the material property is specified. The equation contains a penalization factor that penalizes the intermediate thickness or densities to ensure the physical realizability of elements. In level set approach, a level set function is defined over the design space. The space boundary is specified by the zero-level contour, and the structure is defined by the domain with positive function values. During the optimization process, the level set function is mapped to the mechanical model and is updated constantly until convergence. Evolutionary structural optimization iteratively adds or removes material elements based on the update of finite element analysis results until a steady state is reached. The iterative process is controlled by rejection rate and evolution rate. By formulating the design problem as an optimization problem, TO provides a theoretical guarantee on design performance. Some of the main challenges for TO-based methods, as pointed out by Sigmund and Maute [67], are the computational complexity and the generalizability: The computational complexity of optimizing in 3D design space has limited many TO algorithms to 2D problems, while the requirement of deriving the analytical forms of the objective, and the constraints can restrain TO from a wider variety of design problems where the objective function has a complicated or implicit form. In more recent work, deep learning-based approaches are used to address the computational challenge. Guo et al. [68] mitigates the design complexity of TO by first constructing lower-resolution designs and then converting them to higher-resolution designs using a neural network.

An alternative way of creating new designs with improved performance is by using machine learning-based approaches. In this type of approach, a machine learning model learns to generate new designs from existing designs used as training data. The enhancement of generated designs can be achieved by updating the machine learning model or the training data set. Compared with TO, a machine learning-based approach requires minimal domain knowledge (i.e., the formula of material stiffness to define the objective function) and can therefore be implemented as a highly automated procedure. The problem formulation can be characterized by data samples instead of an analytical expression. As a tradeoff, it is usually difficult for this type of approach to provide a theoretical guarantee of the design performance. Recent work proposed by Oh et al. [69] proposes a deep generative design framework that incorporates both TO and GANs for design generation, where the GAN-generated designs are used as baseline design for TO, and the TO-generated designs are used as training data for GANs. This interesting combination of TO and GANs helps to ensure the quality of the generated designs on both performance and visual appearance and is considered by the authors as a potential direction for future work.

Machine learning is starting to be used to augment the design optimization process by learning how numerical fields influence design decisions [70]. The method presented in this work seeks to teach machine-learned salient features of design in order to enable exploration of a variety of concepts. Such capabilities have the potential to augment the capacity of designers to create designs that exist beyond the training data set. In essence, both the generation (i.e., using the deep generative model) and training set updating (i.e., using the physics-based simulation model) are considered a "black box" so that the model learns the relationship between form and function, with minimal input from the designer. The method to achieve this is discussed next.

3 Method

This work presents a novel, self-updating generative design model using physics simulation. This iterative process is composed of the components shown in Fig. 3 as is described in several steps including the acquisition/curating of training data, deep learning model training, simulation and evaluation of sampled designs, post-processing and filtering of the new generated data set, and retraining or iteration. The method seeks to have the generative design model to enhance the quality of its design by getting feedback from an evaluation process for its generated designs.

The authors hypothesize that training data that includes machine-validated designs from a physics-based virtual environment increases the probability of generative models creating functionally feasible design concepts. The hypothesis is posed such that at each iteration of our framework, the mean performance is statistically different, formally stated as

$$H_0{:}\mu_i = \mu_1$$
$$H_a{:}\mu_i \neq \mu_1$$

where μ_i ($i = 2, 3\ldots I$) is the population mean for the ith design iteration and I represents the number of total iterations for the hypothesis test.

A sufficiently large sample size ($n \approx 1000$) is attained in order to invoke the Central Limit Theorem, thereby supporting the assumptions made in the Z-test [71]. Testing this hypothesis will reveal whether the impact that the physics-based simulation model has a

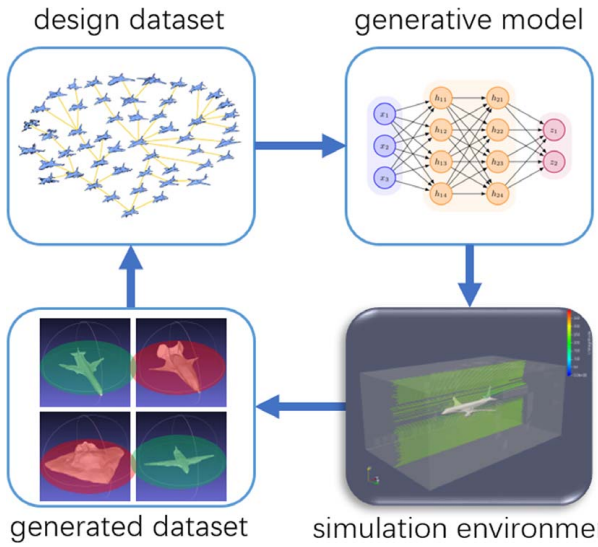


Fig. 3 The flow of this method, which iteratively populates a data set with generated objects. Each run start with a design data set consisting of design concept data (top-left plot). A generative neural network model (top-right) is trained using the design data set to generate new data. Samples of the generated data are sent to a physics-based simulation environment for performance evaluation (bottom-right). The generated data samples are sorted by the performance evaluate results (bottom-left). The best samples are selected to replace a part of the design concepts in the design data set. The updated design data set is used to retrain the generative neural network, thus completing the design-evaluation loop.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

positive effect on enhancing the quality of the computer-generated designs. The knowledge gained by this test will reveal the ability to penalize the GAN-generated designs that contain flaws (e.g., improper shape of the fuselage which intent to cause more drag force when flying in the air.)

3.1 Acquisition of Training Data. The initial training requires a repository of 3D objects. In general, these 3D objects can take any commonly used forms such as 3D meshes, 3D point clouds, and voxels. Since the generative adversarial network used in this paper is designed to generate point clouds, the 3D models in the repository need to allow external surface points to be extracted or approximated from the model surface. The 3D coordinates of the surface point are used to define a matrix $\boldsymbol{M} \in \mathbb{R}^{n\times 3}$ where n denotes the number of points. If the value of n varies among objects, a process such as down-sampling (by randomly selecting a fixed number of points out of n points in the original model) is required to fix n to some constant $\hat{n}$ for all 3D objects in the repository. All point-cloud models from the repository are normalized to ensure the quality of data generation. The normalization process ensures that all 3D models are scaled to have the same size in a particular dimension (e.g., the x-axis) and are placed in the same position and orientation (e.g., in a data set of aircraft models, all aircrafts have their geometric center at the origin and head toward the positive direction of the y-axis.)

3.2 Generative Adversarial Network Model Training and Generation. The problem of 3D model generation is formulated as a problem of generating the external surface points of a 3D model. A GAN model is formulated to generate the surface points. Let $\boldsymbol{M_r} \in \mathbb{R}^{\hat{n}\times 3}$ denote a point-cloud model from the training data set. Following the convention of many GAN-related problem formulations, the models in the training data set are considered to satisfy a probability distribution. Denote the density of such probability distribution as $p(\boldsymbol{M_r})$. By creating a neural network that generates surface points, a parametric probability distribution $p_{\boldsymbol{\Theta}}(\boldsymbol{M_z})$ with parameter set $\boldsymbol{\Theta}$ is introduced to approximate $p(\boldsymbol{M_r})$, where $\boldsymbol{M_z} \in \mathbb{R}^{\hat{n}\times 3}$ denotes a point-cloud model generated by GAN. Thus, the process of training the neural network is essentially a process of searching for the optimal value of network parameters in $\boldsymbol{\Theta}$, such that the difference between $p(\boldsymbol{M_r})$ and $p_{\boldsymbol{\Theta}}(\boldsymbol{M_z})$ (e.g., the Jensen–Shannon divergence) is minimized. To learn $p(\boldsymbol{M_r})$, consider another probability distribution $p_z(\boldsymbol{z})$, where $\boldsymbol{z} \in \mathbb{R}^m$ is an m-dimensional random vector named the latent variable with probability distribution chosen by the user. The neural network is constructed as a parametric function $f:\mathbb{R}^m \rightarrow \mathbb{R}^{\hat{n}\times 3}$ that transforms $p_z(\boldsymbol{z})$ to $p_{\boldsymbol{\Theta}}(\boldsymbol{M_z})$. The neural network that maps $\boldsymbol{z}$ to $\boldsymbol{M_z}$ is the generator network of GAN. In general, the surface of a 3D object forms a non-Euclidian space, where the Euclidian distance between surface points in the 3D space does not necessarily reflect their proximity along the 2D surface. As a result, it is difficult to define an order in $\boldsymbol{M_z}$ such that the indices of its row vectors are highly correlated to the proximity of points along the object surface. To further illustrate this difficulty, consider a point on the 3D object surface. A neighborhood of this point can be defined as the collection of all surrounding surface points whose distance along the surface is less than a threshold value. This neighborhood indicates spatial proximity of points on the surface. It would be ideal if this information can be captured by convolutional layers in the discriminator for the purpose of geometric feature extraction. However, when these surface points are represented by row vectors in $\boldsymbol{M_z}$, for an arbitrary neighborhood of a surface point, it is not feasible to arrange the row sequence in $\boldsymbol{M_z}$, such that rows belonging to the same neighborhood are close to each other. Another example that illustrates the challenge in geometric feature extraction is the following: When the row vectors in $\boldsymbol{M_z}$ are shuffled, $\boldsymbol{M_z}$ becomes a different matrix to the discriminator, although it still represents the same geometry in the 3D space. Since it is difficult to find a proper sequence for the row vectors in $\boldsymbol{M_z}$ that clearly reflect the geometric feature of the point cloud, the authors choose not to impose regularity to the sequence of elements in $\boldsymbol{z}$ or $\boldsymbol{M_z}$, but to employ design for the discriminator that is insensitive to point sequence variation. For generator design, a fully connected neural network model is used, with ReLU used as the activation function for the first three layers and tanh used as the activation function for the last layer. The main motivation for choosing fully connected layers instead of convolutional layers is as follows: convolutional layers are good at making use of the spatial patterns in an input array for feature extraction. In the generator model, the input is a latent variable defined in some lower-dimensional vector space. Prior to the design of the GAN, the relationship between vector entries in the latent variable is unknown. Therefore, the authors choose to be conservative and use fully connected layers, which do not take advantage of any presumed spatial patterns in the input array. The discriminator network adopts the classifier design in Ref. [72], as such a neural network design has shown strong performance in extracting geometric features from point-cloud matrix representations. The reason for the effectiveness of this classifier is that its structure is designed to extract the geometric feature that is invariant of the point sequence in the input point-cloud matrix representation. Once this feature is extracted, regular network structure such as fully connected layers can be used as a classifier that classify the features of true point clouds from the features of the fake point cloud. The transformation from the original point-cloud matrix space to the point sequence-invariant matrix space is performed in two steps. First, a multiplicative transformation using a mini-network is applied to the input point-cloud matrix, in order to make the input point-cloud invariant of certain geometric transformations such as rigid transformation. This step can be represented as follows:

$$\boldsymbol{M_{out}} = \boldsymbol{M_{in}} f_T(\boldsymbol{M_{in}}) \tag{3}$$

where

- f_T is a function implemented by a mini-network whose structure resembles the larger discriminator network;
- $\boldsymbol{M_{in}} \in \{\boldsymbol{M_r}, \boldsymbol{M_z}\}$ is the input of the discriminator; and
- $\boldsymbol{M_{out}}$ is the output of the multiplicative transformation.

In terms of the aircraft case study, $\boldsymbol{M_{in}}$ is an aircraft point cloud from either the generator or the training data set, and $\boldsymbol{M_{out}}$ is a matrix that represents extracted features from the function implemented by the mini-network f_T. Second, a transformation is applied to each row vector of $\boldsymbol{M_{out}}$ in order to create the sequence-invariant feature vector of the input point cloud, as shown in the following equation:

$$f_i = h_{mlp}(\tilde{\boldsymbol{p}}^{(i)}), \quad i = 1, 2, \ldots, N \tag{4}$$

where

- $\tilde{\boldsymbol{p}}^{(i)}$ is the ith row of $\boldsymbol{M_{out}}$;
- f_i is the ith element of the feature vector; and
- h_{mlp} is the function for row vector transformation approximated by a series of fully connected layers.

The overall structure of the discriminator is shown in Fig. 4, which is a modified version of Ref. [72] that omits several of the initial layers to enhance the computational efficiency of the training model, with minimal performance decrease. The block of input transform corresponds to the multiplicative transformation, the blocks "$mlp1$," "$mlp2$," …, "$mlp2500$" represent the transformation of row vectors, and the true/fake classifier classifies the feature vectors into true data or fake data. An overview of the combined framework of point generating GAN and mesh constructor network can be seen in Fig. 5. To illustrate the specialties of the GAN design in this paper, a comparison of the authors' design with several recently proposed approaches is shown in Table 2.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

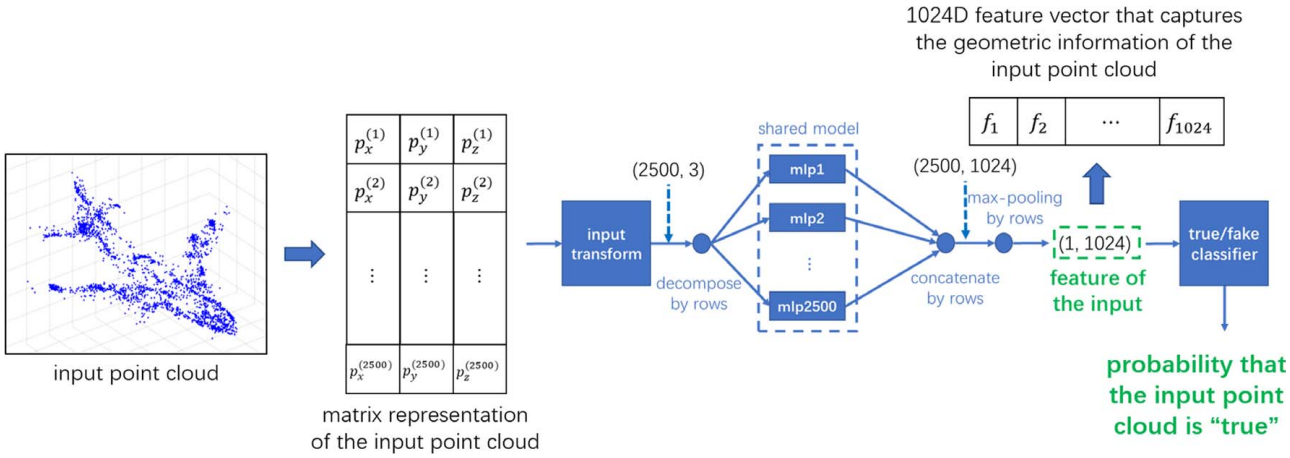


Fig. 4 Design of the discriminator

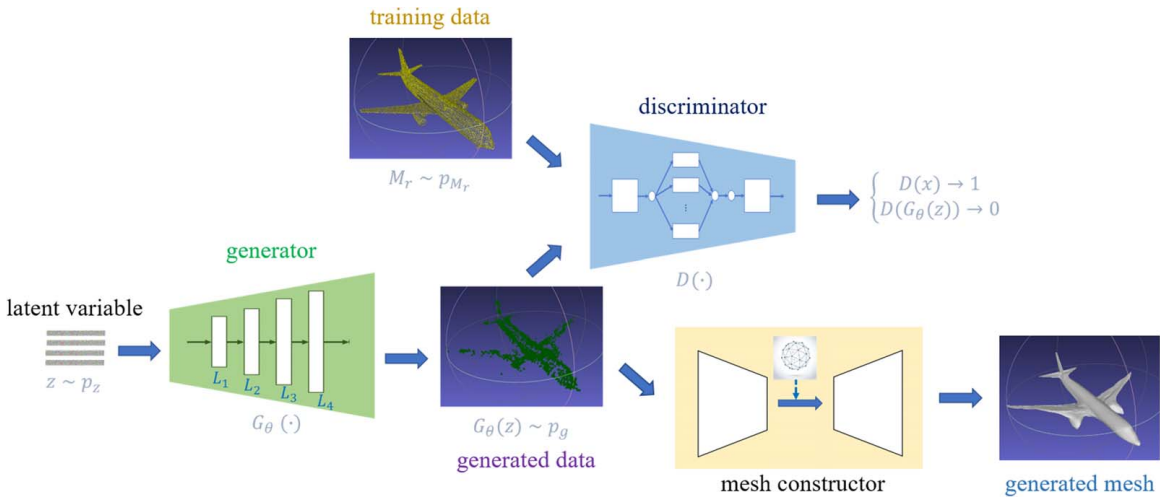


Fig. 5 Generative network that generates aircraft models from samples of normally distributed random variables sampled from the latent space

Table 2 Summary of several design attributes of the authors' GAN design and recent generative models for 3D object generation

Attributes of designs	Generative network designs					
	3D GAN [30]	Ben-Hamu et al. [31]	Tan et al. [32]	Shape VAE [35]	Point cloud GAN [37]	Authors' design
Network type	GAN	GAN	VAE	VAE	GAN	GAN
Inference from the given model required	No	No	Yes	Yes	Yes	No
Generated data format	Voxel	Mesh	Mesh	Point cloud	Point cloud	Point cloud
Post-processing for 3D model manufacturability	No	No	No	Mesh reconstruction	No	Mesh reconstruction
Assumption of pointwise correspondence	No assumption	Required for a few points in a training 3D model	Required for all points in a training 3D model	No assumption	No assumption	No assumption

Physics-based evaluation tools such as OPENFOAM and SOLIDWORKS typically use FEA to simulate forces and moments in a virtual environment.

Therefore, a mesh model or a voxel model are required over a point-cloud model for physics-based evaluation, as they are better supported by evaluation tools using FEA. In this paper, a pretrained neural network is used to convert a generated point-cloud model to a mesh model. The neural network mesh constructor follows the design of an autoencoder proposed in Ref. [34], which uses the 3D coordinates of the points in a point-cloud model to morph a spherical mesh model. Specifically, the spherical mesh model is defined as a tuple $S = \{V, F_m\}$, where V is the set of vertices and F_m is the set of faces. The encoder block of the autoencoder takes the point-cloud model $\boldsymbol{M}$ as input and outputs a k-dimensional feature vector $\boldsymbol{x} \in \mathcal{R}^k$. Let $\boldsymbol{v} \in \mathcal{R}^3$ represent the 3D coordinates of a vertex V. A new vector, $\boldsymbol{p} \in \mathcal{R}^{k+3}$, can be defined by stacking the two vectors $\boldsymbol{x}$ and $\boldsymbol{v}$. Using $\boldsymbol{p}$ as the input to the decoder, a point-cloud model $\tilde{\boldsymbol{M}}$ can be generated by the autoencoder. Let $\tilde{V}$ be the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

set of vertices from $\tilde{\boldsymbol{M}}$, then a mesh model $\tilde{S} = \{\tilde{V},\ F_m\}$ can be constructed from $\boldsymbol{M}$. $\tilde{S}$ is a transform of S such that the vertex indices in $\tilde{S}$ are the same as in S, while the vertex coordinates are changed using the values in $\tilde{\boldsymbol{M}}$. The motivation for using an autoencoder to construct the mesh model is that the autoencoder can be trained to morph the spherical mesh, such that the morphed mesh can approximate the shape of the point cloud.

3.3 Physics-Based Evaluation and Model Retraining. Generated concepts from the generator network are evaluated next in a simulation environment to determine whether they would adequately perform a function that an object of this class should be able to perform (e.g., an aircraft that generates sufficiently low drag force in the air). This method considers the case of a CFD simulation, in which the simulation must accurately characterize the interaction of the design with fluid flow in the simulation environment.

Once the generated designs are tested in a simulation environment, each design receives a performance score r defined as the inverse of the drag coefficient computed by OPENFOAM. This definition is chosen so that a higher value of r corresponds to a lower drag coefficient value. After sorting all the generated designs according to their performance scores, the top ε-percent designs by performance will be selected as the functionally feasible or successful designs. The value of ε can be assigned by the human designer, depending on the functional needs of the design.

Next, the ε-percent successful designs are used to randomly replace the same number of designs in the training data set $\boldsymbol{T}$ to form a new data set for GAN training. At Iteration 1, the training data set $\boldsymbol{T}$ only contains the human-generated instances $\boldsymbol{H}$. At iteration $i > 1$, a fraction of the training data $\boldsymbol{T}$ is replaced by the designs validated in the physics-based simulation environment. Over time, all instances of the human-generated training data will be replaced with data that have been validated in the simulation environment.

In the case of ideal training, the probability distributions of the training data and the generated data become identical after the first iteration of training. Thus, any generated data point is merely a sample from the same probability distribution that characterizes the initial training data set. However, in practice, there is a discrepancy between the distributions of the training data and the generated data. Furthermore, since the training data set defines a sampled distribution with finite number of samples, replacing a part of these samples with generated data points of higher performance has the potential to change the probability distribution of the training data set and result in higher quality GAN output. The results that test this hypothesis are presented in Sec. 4.

4 Application

To test the hypothesis stated in Sec. 3, on 3D generative designs, a case study is performed. The case study considers the problem of generating and evaluating 3D aircraft models. A GAN is designed to generate 3D aircraft designs. The generated designs are sent to a CFD analysis tool named OPENFOAM for performance evaluation. The performance of a design is quantified by the inverse of its drag coefficients computed in OPENFOAM. The motivation for choosing the drag coefficient is that it provides an evaluation on the fuel-economy of an aircraft design: a design with less drag force allows more fuel to be used to generate lift force and forward velocity rather than counteracting the resistance from the airflow.

4.1 Acquisition of Training Data. The initial training requires a repository of 3D objects. In this paper, the ShapeNet database [73] is used to initially train the deep learning model. ShapeNet is a developing data set of 3D shapes which is popular among researchers in 3D model processing and other related disciplines. The data set contains approximately 51,300 unique models divided into multiple categories, where Category No. 02691156 is

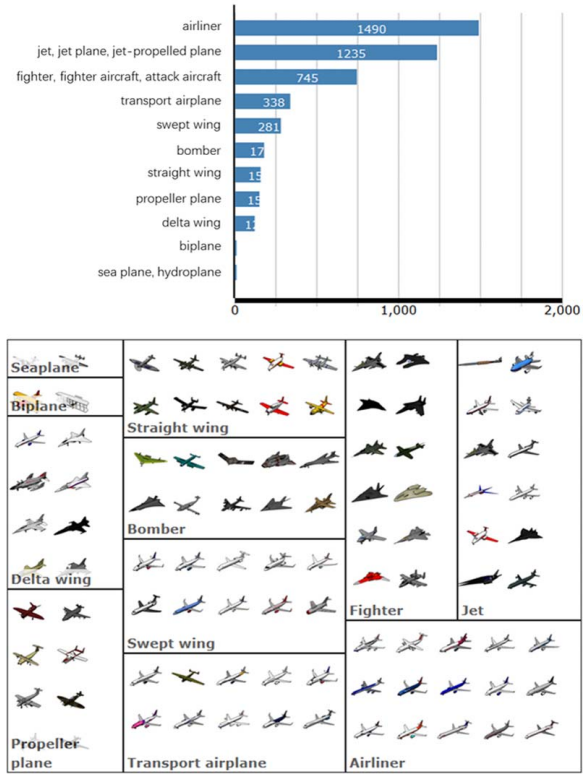


Fig. 6 Model taxonomy of ShapeNet Model category No. 02691156: top subplot, model statistics by classes, and bottom subplot, visualization of model samples from each class

chosen as the training data set for neural network training. This training data set contains 4045 3D models of aircraft with breakdown of the types shown in Fig. 6.[1] Each model is described using the OBJ geometry definition, which is a tessellated representation containing surface points and the corresponding surface normal vectors. Only surface points are used for neural network training, i.e., a point-cloud representation (connectivity is addressed later). Down-sampling is applied to the original surface points such that the number of points in each model is fixed to 2500.

Although most models in this data set come from real-world aircraft, they may be designed to serve different functional purposes. As a result, when tested in a fixed performance evaluation environment (e.g., a wind tunnel test with fixed flow velocity and angle of attack), these models do not necessarily produce similar performance scores. If training a generative neural network with such a data set, the generated aircraft models are expected to vary in performance, as has been observed in the computer experiment for aircraft model evaluation using OPENFOAM. Note that it is this variation in performance that creates room for quality enhancement of model generation by the GAN model.

4.2 Generative Adversarial Network Training and Generation. As shown in Fig. 4, each generated model or training model is described by the 3D coordinates of 2500 surface points. The latent variable is chosen as a 100-dimensional normally distributed random vector. The four fully connected layers of the generator, denoted as "L_1" to "L_4," are chosen to be (256, 512, 1025, and 7500), where the number of layers and the size of the first three layers are determined after experimenting with various neural network structures. The size of the fourth layer corresponds to the number of entries in a 2500-by-3 matrix that represents the generated point cloud. The optimization solver for network training is

[1] https://www.shapenet.org/taxonomy-viewer

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

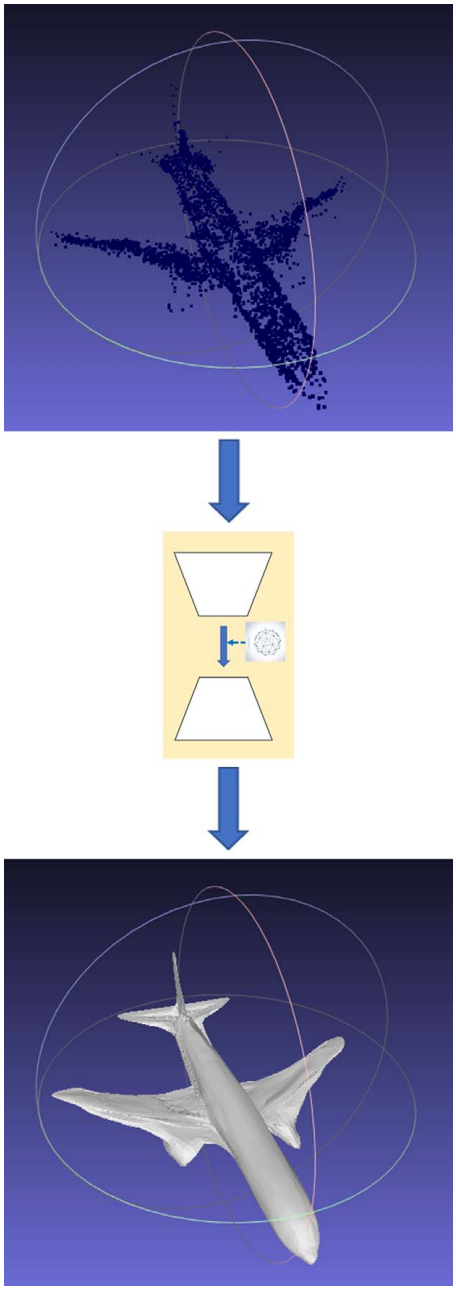


Fig. 7 Process of mesh construction: the mesh constructor neural network (middle) takes a generated point-cloud model (top) as input and outputs a mesh model (bottom)

chosen as AdaGRAD, with learning rate set as 10^{-3} for the first 20 epochs and 10^{-4} for the last 10 epochs. After training the GAN with the initial training data set consisting of ShapeNet models, the generator of the GAN network generates 1080 new aircraft designs. The new designs that initially appear as point clouds are converted to mesh models before being sent to the physics-based evaluation environment for performance evaluation. A spherical mesh of 7446 vertices is used to construct surface mesh from the generated point cloud. As a result, the number of surface points in each 3D model is increased from 2500 to 7440 after mesh reconstruction. An example of a generated point-cloud model and its reconstructed mesh is shown in Fig. 7. More examples of the generated mesh models are shown in Fig. 8.

4.3 Physics-Based Evaluation of Generated Design Concepts. In physics-based design evaluation, the drag coefficient is computed using the open-source CFD library OPENFOAM.[2] OPENFOAM implements polyhedral mesh handling enabling automated meshing and computation against finite volume cells. This automated process evaluates hundreds of design concepts generated by the GAN model. The flow physics used in the OPENFOAM uses the SimpleFoam solver. The solver runs iteratively to solve the Reynolds-Averaged Navier–Stokes equations to compute the force coefficient values, with length-scaled Reynolds number of approximately $\mathrm{Re} \approx 10 - 20 \times 10^6$ and a single angle-of-attack of 10 deg.

In a simulation, a computational region is defined around a nominal design, where airflow is simulated over time. The simulation runs for a fixed number of time steps. The final flow speed and air pressure are used to compute the drag coefficient. The computational region around a nominal design is shown in Fig. 3 (bottom-right). The results of the simulation are used to compute the drag coefficient about the object using the same freestream velocity and reference length/area for each object. The constant values are used so that the deep learning model can learn about the geometry of objects that are all of the similar scale.

Each simulation places the object in the fluid domain, generates a finite volume mesh, and performs 200 iterations of the solver. After 200 iterations, the value of the drag force coefficient converges to a small range. The performance score of the model is calculated as the inverse of the converged drag coefficient. In each run of performance evaluation, 1080 generated models are processed by OPENFOAM to compute their drag coefficient values, wherein the best 405 models in terms of low drag coefficient are considered the functionally feasible or successful designs and are used to randomly replace 405 training models before the next retraining of GAN. The number 405 is empirically chosen to account for 10% of the training data set, and the number 1080 is chosen such that approximately one-third of the generated models are selected for retraining. The quantities 10% and one-third are hyperparameters that affect the performance of the proposed model retraining procedure. If the proportion of replaced training data is too small, it will take too many iterations for the performance to improve. On the other hand, if the proportion of replaced training data is too high, there won't be enough iterations before the training data are completely replaced by the generated data. The values 10% and one-third were chosen to strike a balance between these two cases and should be determined based on the performance distribution of the generated data set.

After the data set is updated with the successful designs from the OPENFOAM evaluation, the GAN is retrained. Each retraining is run for 30 epochs. The retrained GAN is then used to generate 1080 new aircraft designs. The new designs will then be evaluated in OPENFOAM to select the best 405 designs under similar conditions as the ones before them. Thus, the next round of generation–evaluation cycle is ready to start. All CFD evaluations were performed with an Amazon AWS m5a.24xlarge instance. On average, each model was evaluated in about 44 s. Thus, evaluating three iterations of 1080 aircraft designs took 39.6 h.

5 Results and Discussion

5.1 Discussion on Experimental Results. The main investigation of this work is to determine whether the quality of the generated data is improved by retraining the neural network model using new training data set that contains machine-validated designs. The performance scores of the generated data are tracked over iterations of the generation–evaluation cycle, as more validated examples are introduced to the training set. Figure 9 shows the distributions of scores for three iterations, where the distribution in the upper subplot is from models generated by the GAN model after the initial training, the distribution in the middle subplot in Fig. 9 is from models generated by the GAN model after the first retraining,

[2] http://www.openfoam.com

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

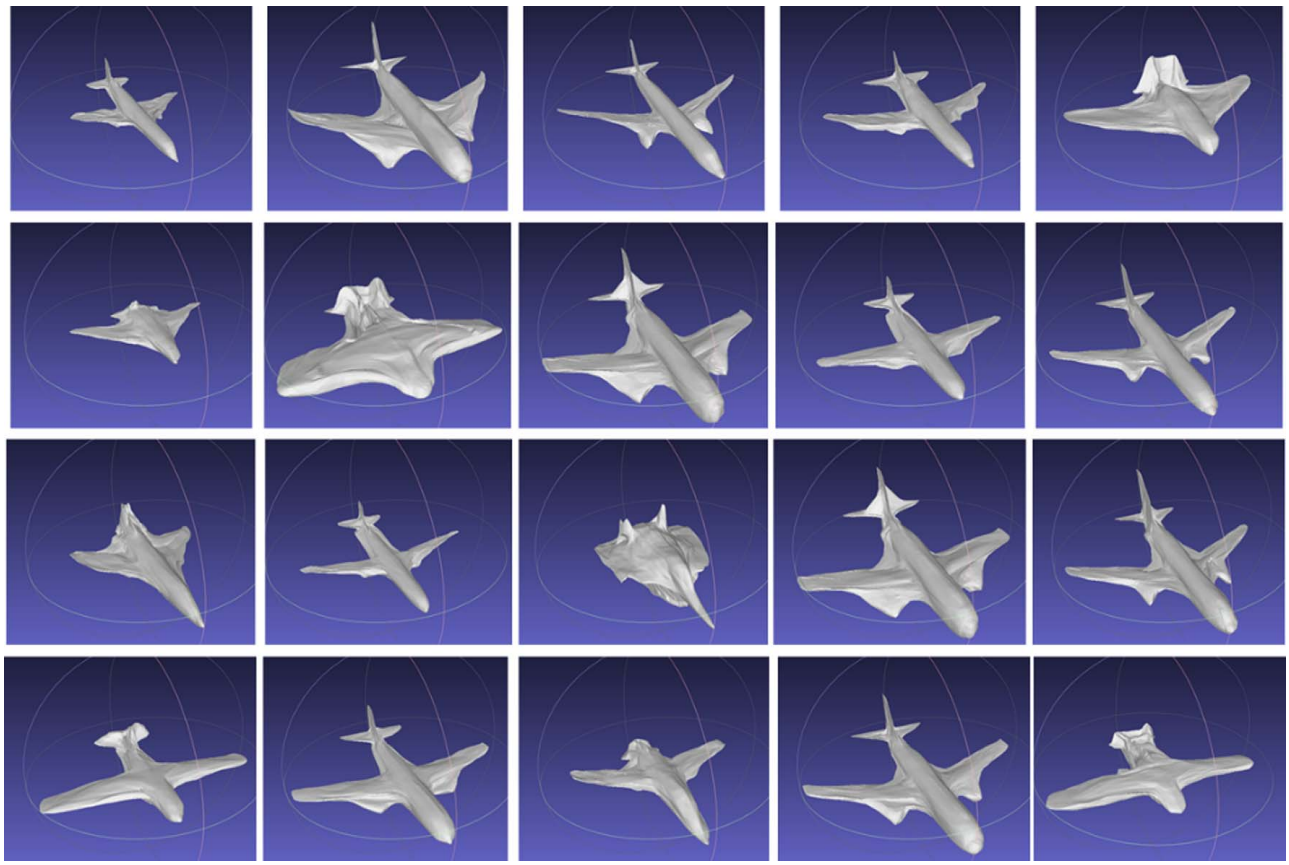


Fig. 8 25 Samples of the generated mesh model

and the distribution in the lower subplot is from models generated by the GAN model after the second retraining. Both the scores after the first retraining, and the scores after the second retraining, show an increase in the mean performance score, compared with the scores after the initial training. In terms of standard deviation, the score distribution after the second retraining has a smaller variance than that after the initial training. In the second retraining, 773 out of the 4045 models in the training data set are replaced with the generated models. The relevant statistics are listed in Table 3, where the results of three Z-tests are shown. The first Z-test reveals whether the score distributions in Iterations 1 and 2 are statistically significantly different. The second Z-test reveals whether the score distributions in Iterations 1 and 3 are significantly different, and the third Z-test reveals whether the score distributions in Iterations 2 and 3 are significantly different. These results are evaluated with a significance level of 0.05. According to these Z-test results, both the score distributions in Iteration 2 and Iteration 3 are significantly different from that of Iteration 1, while the difference between the score distributions in Iteration 2 and in Iteration 3 is not significant. This result indicates that, compared with the generated designs using the initial training data set, the generated designs using the new training data set that contains machine-validated generated designs, have a statistically significant increase in the functional performance.

Another relevant statistic that indicates the quality of the generated models is the proportion of valid designs. Among all generated models sent to OPENFOAM for evaluation, a majority of the models produce a positive drag coefficient value which is in accordance with the real-world practice in aircraft design and tests. However, a small fraction of the generated models either produce a negative drag coefficient value or was rejected by OPENFOAM for being a non-manifold object. Models that meet one of these criteria yield infeasible evaluation results and are considered invalid designs. The existence of invalid designs is due to the fact that a GAN is trained to generate data that resemble the training data in the sense of a probability distribution. As a result, there exists a probability that the generator from the GAN model generates data samples that are quite different from any data sample in the training data set. The proportion of valid design is defined as the number of valid models divided by the number of all models sent to OPENFOAM in a design iteration. After the initial training, the proportion of valid design is 0.9176. This number increases to 0.9454 after the first retraining and slightly reduces to 0.9417 after the second retraining. This result indicates that retraining improves the performance of GAN by increasing the rate of valid model generation. During the automatic evaluation process in the computer experiment, if the evaluation program determines that a mesh model is invalid, it will skip that model and move on to evaluate the next model in the generated data set. The invalid designs with negative coefficient values are filtered out, and the fraction of top-score models are selected to update the training data set during the next iteration.

As a justification for choosing the inverse of drag coefficient as the performance score, generated models sampled from three regions in the performance score distribution of Iteration 3 are examined. The three regions correspond to the area of low scores, the area of mediocre scores, and the area of high scores, respectively. From each region, four model samples are selected for visualization. As shown in Fig. 10, the models with lower scores appear to have a larger volume and a cumbersome shape, while the models with high scores tend to have a more streamlined design and a smaller volume. The models with mediocre scores have an intermediate volume and *look* similar to a regular airliner. The change of model form with respect to the performance score aligns with common intuition for predicting the functionality of the model. This observation result provides support to the validity of choosing the inverse of drag coefficient as the performance score.

In the original training data set, 1490 out of 4045 models in the initial training set are classified as airliners, as shown in Fig. 6.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

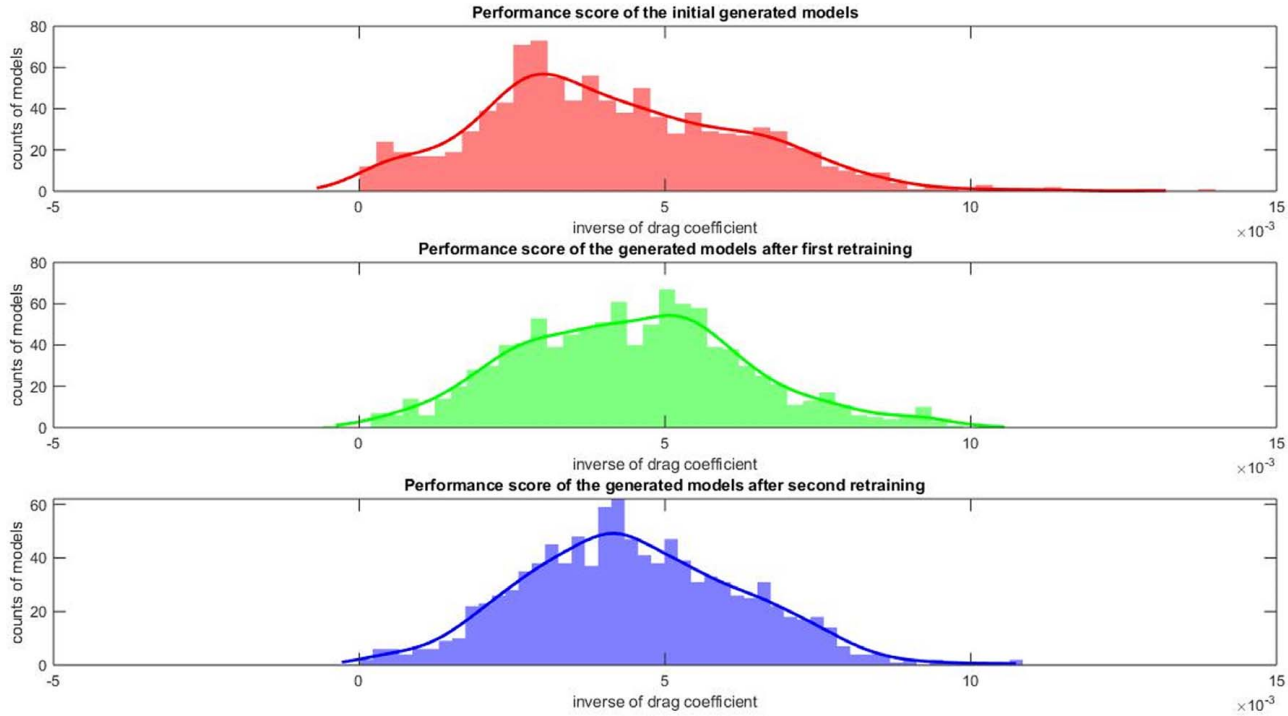


Fig. 9 Distributions of scores for the first three training iterations

Table 3 Summary statistics of the three design iterations

Design iteration	Mean	Standard deviation	*p*-value
1	4.0921×10^{-3}	2.1138×10^{-3}	(iter1 & iter2) 1.6852×10^{-9}
2	4.4907×10^{-3}	2.1526×10^{-3}	(iter1 & iter3) 4.3391×10^{-9}
3	4.4813×10^{-3}	1.7859×10^{-3}	(iter2 & iter3) 0.8883

Note: "(iter1 & iter2)" in the *p*-value box refers to a *z*-test that determines whether score distribution from Iteration 2 is the same as that from Iteration 1. "(iter1 & iter3)" and "(iter2 & iter3)" are interpreted similarly.

Correspondingly, more airliner-type models appear in the generated data, which moves the mean performance score of the generated designs close to the average score of "airliner-like" designs. One common pattern in the airliner type of design is the engines under the wings. In the phase of GAN training where only the form of training data is learned, such a pattern is inherited by the generated models. In the phase of performance evaluation, since no propelling forces are simulated, the engines only increase the drag force on an aircraft model and, hence, become counterproductive factors in pursuing high-performance scores. Ideally, to increase the average performance scores of the generated designs, engines should be removed from any aircraft models in the initial training data set. But due to the lack of engine-free airliners in the available repository and the limited time to modify the initial training data, the authors proceeded in the case study with the existing ShapeNet models. As expected, the generated models without engines perform better than the ones with engines: Fig. 9 shows that none of the four model samples with high scores have an engine like object, while three out of four model samples with mediocre scores have engines. This contrast suggests that the generation–evaluation framework presented in this paper can be applied to make GAN learn to "de-feature" the engines or other parts in a generated form that reduce the function of the design concept.

5.2 Benefits and Limitations of the Proposed Approach. The proposed approach for automatic design is intended to achieve the following two goals: first, the design generation method can learn from a given set of existing designs and infer a set of new designs that have both similarity and novelty, compared to the given designs. Second, the generation process of new designs can be directed by some user-specified objective. The object metric can either be given as labels or be obtained using theoretical analysis, computer simulation, numerical approximation, etc.

While the original ShapeNet database provides feasible and diverse choices that fulfill their intended function(s), users are limited to the given original designs. With a GAN model, the original dataset can be augmented by a set of GAN-generated designs, which provides the users with more choices. Compared with a design in the original data set that takes human designers' time and domain knowledge to develop, a GAN-generated design is generated in less than one second and requires minimal domain knowledge and special design software. Therefore, GAN-based generative design provides an efficient way of expanding an existing set of reference designs.

The mixture of models in a training data set is for GANs to learn the common features and the diversity of a class of designs. In practice, the common features and the diversity of designs are often understood by human designers in an implicit way. Such understanding can be difficult to summarize using explicit mathematical formulas but are needed in automatic design. GANs provide a parametric function approximation of such understandings and, in this way, help to advance design automation.

Although only one label (i.e., aircraft) is used in the computer experiment, the proposed design approach in this paper is not limited to one performance metric. For example, the designer can

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

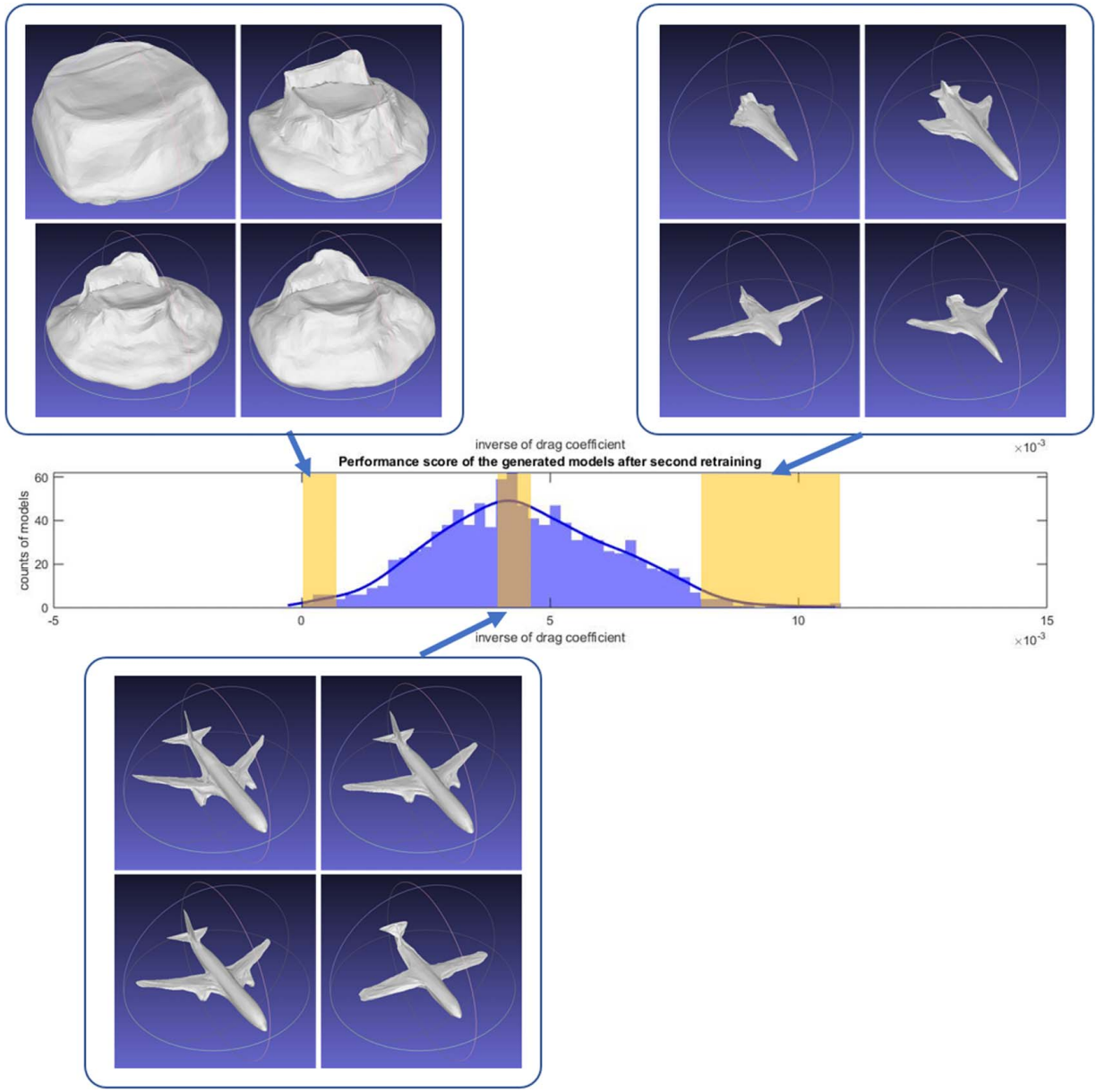


Fig. 10 Visualization of generated models in Iteration 3 sampled from three different performance score regions

first define multiple labels that correspond to different intended functions. Then, the multiple labels can be incorporated into a single performance metric as a weighted combination or other mathematical formulation, where increasing the single performance metric is no longer equivalent to increasing one label, instead seeking a tradeoff between all designed labels.

The instructive and constructive information for aircraft development that the generated data are intended to provide is the various forms of an aircraft along with their associated drag coefficient values. Indeed, the users do not have direct control over the drag coefficient values of the generated designs, and the evaluation procedure to acquire the drag coefficient values can be computationally expensive. The authors have realized this limitation of the proposed approach. To improve the utility of the newly generated database, the authors are working on two approaches as future work. In the first approach, a neural network is used to approximate the performance evaluation conducted by analysis software. With such a neural network, the human engineer can instantly obtain the estimated performance metric of every generated design and select the top few generated designs based on the performance estimation. In the second approach, the traditional GAN model is upgraded to a conditional GAN model. With the conditional GANs, the engineer can specify a range of performance metric value, then the generator will generate a set of new designs whose performance falls within the range with high probability. The proposed approach in this paper serves as a foundation of the two aforementioned approaches under development.

5.3 Comparison With Topology Optimization. Considering the popularity of TO in generative design, a brief review is made on TO-based aircraft design and its difference from the proposed approach. Assume that the objective of TO is to minimize the drag coefficient. Starting from a baseline design, TO can first calculate the stress and velocity of airflow to update the drag force on the aircraft model. Then, with sensitivity analysis, TO can compute the gradient of the drag force with respect to the design variable and have the design variable updated using the gradient. By iteratively running these steps, a design with minimal drag coefficient can be obtained. This procedure has been shown to effectively minimize

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

the drag force of an object in two-dimensional airflow [74], but can be restrained from optimization in 3D space due to computational cost. In comparison, the approach proposed in this paper can efficiently generate a large number of designs in a 3D design space. The time it takes to produce one generated design is less than a second, as the generation is done by a pretrained neural network (the generator of a GAN model). In its defined design space, TO is a rigorous way to improve the performance of the design, while the authors' approach is less rigorous and requires expensive CFD simulation tool for design evaluation. TO starts from a given design and searches for a new design with maximal performance improvement. In comparison, the proposed approach starts with a set of existing designs (the training data set) and generates a set of new designs (the generated data set), where the generation process is governed by the objective of maintaining visual similarity (via the loss function of GANs) and the objective of improving the design performance (via GAN model retraining). As a limitation, the proposed approach imposes the design objectives in a statistical sense. Hence, it is not informative to make a performance comparison between a single design from the training data set and a single design from the generated data set. Considering these differences, the proposed design is more suitable for producing a large number of new preliminary designs for a human engineer as references. TO and the proposed approach can be used together to improve the performance of the generated designs. For example, before evaluation, each GAN-generated design can be functionally improved by TO. There is room for reducing computational costs in both approaches. While the computational cost for high-resolution designs can possibly be mitigated by a neural network in TO, the proposed approach can use a neural network to approximate the evaluation results, such that the computational cost for model evaluation will drop significantly.

6 Conclusions

This work demonstrates a new method of improving generated data quality by incorporating generated data in the training process. It also considers the possible uses of generated data meant to perform a task, something that until now, has been a highly manual process. By finding models that function correctly for a task and refining on those, a generator can be constructed which generates valid objects with a high probability. This is an important advancement toward using machine learning to achieve objective-driven designs. This automated process began with a noisy data set and through repeated generation and validation of designs achieved a model, which generated a higher proportion of performant designs without sacrificing data quality.

Future work could explore the relationship between the value of ε and the performance score distribution of generated designs in the next design iteration. Other ways of defining the performance score could also be explored to see how they would change the generated designs. Alternative definitions for the loss function in GAN training should be investigated for possible improvement on the performance of GANs. In addition, a reinforcement learning approach could be directly integrated with the training procedure to replace OPENFOAM for faster model evaluation. Another approach may consider a more fully conditioned generation method, which a reinforcement model could learn to use, so that it only generates objects which are known to work. Additionally, future work should consider how this method performs as the data set becomes largely synthetic, by measuring visual diversity and functional performance. Future work should also consider greatly increasing the fidelity of the simulation, through characteristics such as materials and weight distribution.

Acknowledgment

The authors would like to acknowledge Haoyuan Meng, Matthew Dering, Zhaohong Lyu, Pranav Jain and Albert Wilson for their contributions to this work. Any opinions, findings, or conclusions found in this paper are those of the authors and do not necessarily reflect the views of the sponsors.

Funding Data

- This research is funded in part by DARPA (Grant No. HR0011-18-2-0008; Funder ID: 10.13039/100000185).

Nomenclature

r = performance score of a design
$\boldsymbol{x}$ = feature vector in the mesh constructor network
$\boldsymbol{z}$ = latent variable in a GAN or VAE model
$\mathbb{R}$ = 1-dimensional real space
D = discriminator function of GAN
G = generator function of GAN
S = spherical mesh
V = set of vertices in the spherical mesh
$\boldsymbol{H}$ = initial training data set
$\boldsymbol{M}$ = matrix of initial point cloud
$\boldsymbol{T}$ = training data set of GAN
$\hat{n}$ = constant representing a fixed number of points in a point-cloud model
$\tilde{\boldsymbol{z}}$ = new latent variable obtained from linear combination of existing latent variables
f_i = ith element of the feature vector in the discriminator
f_T = function of mini-network in the discriminator
h_{mlp} = function for row vector transformation in the discriminator
$\boldsymbol{z}_1$ = first latent variable sample in the GAN model
$\boldsymbol{z}_2$ = second latent variable sample in the GAN model
F_m = set of faces in the spherical mesh
H_0 = null hypothesis
H_a = alternative hypothesis
$\boldsymbol{M_{in}}$ = input point-cloud matrix of the discriminator
$\boldsymbol{M_{out}}$ = output matrix of the input transformation block in the discriminator
$\boldsymbol{M_r}$ = matrix of point-cloud model from training data set of GAN
$\boldsymbol{M_z}$ = matrix of generated point cloud
$\tilde{\boldsymbol{p}}^{(i)}$ = ith row vector of $\boldsymbol{M_{out}}$
$\mathbb{R}^n$ = n-dimensional real space
$\mathbb{R}^{m\times n}$ = $m \times n$ dimensional real space
$p_{\boldsymbol{\Theta}}(\boldsymbol{M_z})$ = probability density function of the generated point-cloud data
$p_z(\boldsymbol{z})$ = probability density function of the latent variable $\boldsymbol{z}$
$p(\boldsymbol{M_r})$ = probability density function of the point-cloud data for training
Re = Reynolds number
ReLU = rectifier activation function
tanh = hyperbolic tangent activation function
λ = scalar to control the value of $\tilde{\boldsymbol{z}}$
$\boldsymbol{\Theta}$ = set of generator parameters in the GAN model
$\boldsymbol{\mu_i}$ = mean performance score of the generated designs from Iteration i

References

[1] Burnap, A., Liu, Y., Pan, Y., Lee, H., Gonzalez, R., and Papalambros, P. Y., 2016, "Estimating and Exploring the Product Form Design Space Using Deep Generative Models," 42nd Design Automation Conference, Charlotte, NC, Aug. 21–24, Vol. 2A, p. V02AT03A013.
[2] Dering, M., and Tucker, C., 2017, "Generative Adversarial Networks for Increasing the Veracity of Big Data," 2017 IEEE International Conference on Big Data, Boston, MA, Dec. 11–14, pp. 2595–2602.
[3] Dering, M. L., and Tucker, C. S.,2017, Implications of Generative Models in Government, Reports of the 2017 AAAI Fall Symposium Series, Nov. 9–11, pp. 158–163.
[4] Tran, D., Bourdev, L., Fergus, R., Torresani, L., and Paluri, M., 2015, "Learning Spatiotemporal Features With 3D Convolutional Networks," IEEE International Conference on Computer Vision, Santiago, Chile, Dec. 13–16, pp. 4489–4497.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

[5] Chan, T.-H., Jia, K., Gao, S., Lu, J., Zeng, Z., and Ma, Y., Dec. 2015, "PCANet: A Simple Deep Learning Baseline for Image Classification?," IEEE Trans. Image Process., **24**(12), pp. 5017–5032.

[6] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., 2014, "Generative Adversarial Nets," Advances in Neural Information Processing Systems 27, Montréal, Canada, Dec. 8–13, pp. 2672–2680.

[7] Denton, E. L., Chintala, S., and Fergus, R., 2015, "Deep Generative Image Models Using a Laplacian Pyramid of Adversarial Networks," Advances in Neural Information Processing Systems, pp. 1486–1494.

[8] Venugopalan, S., Xu, H., Donahue, J., Rohrbach, M., Mooney, R., and Saenko, K., 2015, "Translating Videos to Natural Language Using Deep Recurrent Neural Networks," The 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Denver, CO, May 31–June 5.

[9] Zheng, S., Jayasumana, S., Romera-Paredes, B., Vineet, V., Su, Z., Du, D., Huang, C., and Torr, P. H., 2015, "Conditional Random Fields as Recurrent Neural Networks," 2015 IEEE International Conference on Computer Vision, Washington, DC, Dec. 7–13, pp. 1529–1537.

[10] Maier, J. R., Fadel, G. M., and Battisto, D. G., 2009, "An Affordance-Based Approach to Architectural Theory, Design, and Practice," Desi. Stud., **30**(4), pp. 393–414.

[11] Ferguson, S., Siddiqi, A., Lewis, K., and de Weck, O. L., 2007, "Flexible and Reconfigurable Systems: Nomenclature and Review," ASME 2007 International Design Engineering Technical Conferences & Computers and Information in Engineering Conference, Las Vegas, NV, Sept. 4–7, Vol. 6, pp. 249–263.

[12] Umeda, Y., Ishii, M., Yoshioka, M., Shimomura, Y., and Tomiyama, T., 1996, "Supporting Conceptual Design Based on the Function-Behavior-State Modeler," Ai Edam, **10**(4), pp. 275–288.

[13] Kang, S. W., and Tucker, C., 2016, "An Automated Approach to Quantifying Functional Interactions by Mining Large-Scale Product Specification Data," J. Eng. Des., **27**(1–3), pp. 1–24.

[14] Christensen, B. T., and Ball, L. J., 2016, "Dimensions of Creative Evaluation: Distinct Design and Reasoning Strategies for Aesthetic, Functional and Originality Judgments," Desi. Stud., **45**(Special Issue: Design Review Conversations), pp. 116–136.

[15] Bohm, M. R., Stone, R. B., Simpson, T. W., and Steva, E. D., 2008, "Introduction of a Data Schema to Support a Design Repository," Comput.-Aided Des., **40**(7), pp. 801–811.

[16] Dering, M., Cunningham, J., Desai, R., Yukish, M. A., Simpson, T. W., and Tucker, C. S., 2018, "A Physics-Based Virtual Environment for Enhancing the Quality of Deep Generative Designs," ASME 2018 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Quebec City, Canada, Aug. 26–29.

[17] Bengio, Y., Mesnil, G., Dauphin, Y., and Rifai, S., 2013, "Better Mixing Via Deep Representations," The 30th International Conference on Machine Learning, Atlanta, GA, June 16–21, pp. 552–560.

[18] Gurumurthy, S., Kiran Sarvadevabhatla, R., and Venkatesh Babu, R., 2017, "Deligan: Generative Adversarial Networks for Diverse and Limited Data," The IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, July 21–26, pp. 166–174.

[19] Ulu, N. G., and Kara, L. B., 2015, "DMS2015-33: Generative Interface Structure Design for Supporting Existing Objects," J. Vis. Lang. Comput., **31**(Special Issue on DMS2015), pp. 171–183.

[20] Andrade, D., Harada, M., and Shimada, K., 2017, "Framework for Automatic Generation of Facades on Free-Form Surfaces," Front. Archit. Res., **6**(3), pp. 273–289.

[21] Whiting, M., Cagan, J., and LeDuc, P., 2017, "Automated Induction of General Grammars for Design," *Design Computing and Cognition '16*, J. S. Gero, ed., Springer, Cham, Switzerland, pp. 267–278.

[22] Pu, Y., Gan, Z., Henao, R., Yuan, X., Li, C., Stevens, A., and Carin, L., 2016, "Variational Autoencoder for Deep Learning of Images, Labels and Captions," Advances in Neural Information Processing Systems 29, Barcelona, Spain, Dec. 5–10, pp. 2352–2360.

[23] Genevay, A., Peyré, G. and Cuturi, M., 2017, "GAN and VAE from an Optimal Transport Point of View," http://arxiv.org/abs/1706.01807.

[24] Kingma, D. P., and Welling, M., 2014, "Auto-Encoding Variational Bayes," 2nd International Conference on Learning Representations, Banff, Canada, Apr. 14–16.

[25] Arjovsky, M., Chintala, S., and Bottou, L., 2017, "Wasserstein Generative Adversarial Networks," The 34th International Conference on Machine Learning, Sydney, Australia, Aug. 6–11.

[26] Chen, W., Fuge, M., and Chazan, J., 2017, "Design Manifolds Capture the Intrinsic Complexity and Dimension of Design Spaces," ASME J. Mech. Des., **139**(5), p. 051102.

[27] Dosovitskiy, A., Springenberg, J. T., Tatarchenko, M., and Brox, T., 2017, "Learning to Generate Chairs, Tables and Cars With Convolutional Networks," IEEE Trans. Pattern Anal. Mach. Intell., **39**(4), pp. 692–705.

[28] Cang, R., Vipradas, A., and Ren, Y., 2017, "Scalable Microstructure Reconstruction with Multi-Scale Pattern Preservation," ASME 2017 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Cleveland, OH, Aug. 6–9, American Society of Mechanical Engineers, New York, p. V02BT03A010.

[29] Cang, R., Xu, Y., Chen, S., Liu, Y., Jiao, Y., and Ren, M. Y., 2017, "Microstructure Representation and Reconstruction of Heterogeneous Materials Via Deep Belief Network for Computational Material Design," ASME J. Mech. Des., **139**(7), p. 071404.

[30] Wu, J., Zhang, C., Xue, T., Freeman, B., and Tenenbaum, J., 2016, "Learning a Probabilistic Latent Space of Object Shapes Via 3d Generative-Adversarial Modeling," Advances in Neural Information Processing Systems 29, Barcelona, Spain, Dec. 5–10, pp. 82–90.

[31] Ben-Hamu, H., Maron, H., Kezurer, I., Avineri, G., and Lipman, Y., 2018, "Multi-Chart Generative Surface Modeling," ACM Transactions on Graphics, **37**(6). Article No. 215.

[32] Tan, Q., Gao, L., Lai, Y. K., and Xia, S., 2018, "Variational Autoencoders for Deforming 3d Mesh Models," The IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, June 18–22, pp. 5841–5850.

[33] Gao, L., Lai, Y. K., Liang, D., Chen, S. Y., and Xia, S., 2016, "Efficient and Flexible Deformation Representation for Data-Driven Surface Modeling," ACM Trans. Graphics (TOG), **35**(5), p. 158.

[34] Groueix, T., Fisher, M., Kim, V. G., Russell, B. C., and Aubry, M., 2018, "A Papier-Mâché Approach to Learning 3D Surface Generation," The IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, June 18–22, pp. 216–224.

[35] Nash, C., and Williams, C. K., 2017, "The Shape Variational Autoencoder: A Deep Generative Model of Part-Segmented 3D Objects," Comput. Graphics Forum, **36**(5), pp. 1–12.

[36] Achlioptas, P., Diamanti, O., Mitliagkas, I., and Guibas, L., 2017, "Learning Representations and Generative Models for 3d Point Clouds," Thirty-fifth International Conference on Machine Learning, Stockholm, Sweden, July 10–15.

[37] Li, C. L., Zaheer, M., Zhang, Y., Poczos, B., and Salakhutdinov, R., 2019, "Point Cloud Gan," ICLR Workshop on Deep Generative Models for Highly Structured Data, New Orleans, LA, May 6–9.

[38] Clayton, M. J., Teicholz, P., Fischer, M., and Kunz, J., 1999, "Virtual Components Consisting of Form, Function and Behavior," Autom. Constr., **8**(3), pp. 351–367.

[39] Umeda, Y., Kondoh, S., Shimomura, Y., and Tomiyama, T., 2005, "Development of Design Methodology for Upgradable Products Based on Function–Behavior–State Modeling," Ai Edam, **19**(3), pp. 161–182.

[40] Crilly, N., Moultrie, J., and Clarkson, P. J., 2004, "Seeing Things: Consumer Response to the Visual Domain in Product Design," Desi. Stud., **25**(6), pp. 547–577.

[41] Balduzzi, F., Ferrara, G., Babbini, A., and Pratelli, G., 2012, "CFD Evaluation of the Pressure Losses in a Reciprocating Compressor: A Flexible Approach," ASME 2012 11th Biennial Conference on Engineering Systems Design and Analysis, Nantes, France, July 2–4, American Society of Mechanical Engineers, New York, pp. 63–72.

[42] Jeong, H. Y., Ha, K. S., Kwon, Y. M., Lee, Y. B., Hahn, D., Cahalan, J. E., and Dunn, F. E., 2007, "Evaluation of the Conduction Shape Factor With a CFD Code for a Liquid–Metal Heat Transfer in Heated Triangular Rod Bundles," Nucl. Eng. Des., **237**(6), pp. 648–654.

[43] Conner, M. E., Baglietto, E., and Elmahdi, A. M., 2010, "CFD Methodology and Validation for Single-Phase Flow in PWR Fuel Assemblies," Nucl. Eng. Des., **240**(9), pp. 2088–2095.

[44] Marchesse, Y., Changenet, C., Ville, F., and Velex, P., 2011, "Investigations on CFD Simulations for Predicting Windage Power Losses in Spur Gears," ASME J. Mech. Des., **133**(2), p. 024501.

[45] Krepper, E., Končar, B., and Egorov, Y., 2007, "CFD Modelling of Subcooled Boiling—Concept, Validation and Application to Fuel Assembly Design," Nucl. Eng. Des., **237**(7), pp. 716–731.

[46] Montazeri, H., and Blocken, B., 2013, "CFD Simulation of Wind-Induced Pressure Coefficients on Buildings With and Without Balconies: Validation and Sensitivity Analysis," Building and Environment, **60**, pp. 137–149.

[47] Dye, J. R., Tay, Y. Y., and Lankarani, H. M., 2015, "Development and Application of Planar Computational General-Purpose Constrained Multibody Simulations in Matlab with Simple Graphical/Visualization Capability," ASME 2015 International Mechanical Engineering Congress and Exposition, Houston, TX, Nov. 13–19, American Society of Mechanical Engineers, New York, p. V04BT04A002.

[48] Sam, R., Arrifin, K., and Buniyamin, N., 2012, "Simulation of Pick and Place Robotics System Using Solidworks Softmotion," 2012 International Conference on System Engineering and Technology, Bandung, Indonesia, Sept. 11–12, IEEE, pp. 1–6.

[49] Turrell, M. D., Stopford, P. J., Syed, K. J., and Buchanan, E., 2004, "CFD Simulation of the Flow Within and Downstream of a High-Swirl Lean Premixed gas Turbine Combustor," ASME Turbo Expo 2004: Power for Land, Sea, and Air, Vienna, Austria, June 14–17, American Society of Mechanical Engineers, New York, pp. 31–38.

[50] Field, D. A., 2004, "Education and Training for CAD in the Auto Industry," Comput.-Aided Des., **36**(14), pp. 1431–1437.

[51] Rozvany, G. I., 2014, *Topology Optimization in Structural Mechanics*, Springer, New York, Vol. 374.

[52] Zhu, J. H., Zhang, W. H., and Xia, L., 2016, "Topology Optimization in Aircraft and Aerospace Structures Design," Arch. Comput. Meth. Eng., **23**(4), pp. 595–622.

[53] Xia, L., and Breitkopf, P., 2017, "Recent Advances on Topology Optimization of Multiscale Nonlinear Structures," Arch. Comput. Meth. Eng., **24**(2), pp. 227–249.

[54] Kanno, Y., 2016, "Redundancy Optimization of Finite-Dimensional Structures: Concept and Derivative-Free Algorithm," J. Struct. Eng., **143**(1), p. 04016151.

[55] Mohammadi, B., and Pironneau, O., 2010, *Applied Shape Optimization for Fluids*, 2nd ed., Oxford University Press, New York.

[56] Borrvall, T., and Petersson, J., 2003, "Topology Optimization of Fluids in Stokes Flow," Int. J. Numer. Methods Fluids, **41**(1), pp. 77–107.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025

[57] Zhou, S., and Li, Q., 2008, "A Variational Level Set Method for the Topology Optimization of Steady-State Navier–Stokes Flow," J. Comput. Phys., **227**(24), pp. 10178–10195.
[58] Othmer, C., Manosalvas-Kjono, D. E., Jameson, A., and Alonso, J. J., 2017, "Aerodynamic Topology Optimization: Some Observations on Hysteresis in Separated Flows," 23rd AIAA Computational Fluid Dynamics Conference, Denver, CO, June 5–9, p. 4413.
[59] Zegard, T., and Paulino, G. H., 2016, "Bridging Topology Optimization and Additive Manufacturing," Struct. Multi. Optim., **53**(1), pp. 175–192.
[60] Langelaar, M., 2016, "Topology Optimization of 3D Self-Supporting Structures for Additive Manufacturing," Additive Manufacturing, **12**, pp. 60–70.
[61] Guo, X., Zhou, J., Zhang, W., Du, Z., Liu, C., and Liu, Y., 2017, "Self-supporting Structure Design in Additive Manufacturing Through Explicit Topology Optimization," Computer Methods in Applied Mechanics and Engineering, **323**, pp. 27–63.
[62] Brackett, D., Ashcroft, I., and Hague, R., "Topology Optimization for Additive Manufacturing," Proceedings of the Solid Freeform Fabrication Symposium, Aug. 2011, Austin, TX, Vol. 1, pp. 348–362.
[63] Bendsoe, M. P., and Sigmund, O., 2004, *Topology Optimization: Theory, Methods and Applications*, 2nd ed., Springer-Verlag, Berlin, Heidelberg, New York.
[64] Allaire, G., Jouve, F., and Toader, A. M., 2002, "A Level-Set Method for Shape Optimization," C.R. Math., **334**(12), pp. 1125–1130.
[65] Xie, Y. M., and Steven, G. P., 1993, "A Simple Evolutionary Procedure for Structural Optimization," Comput. Struct., **49**(5), pp. 885–896.
[66] Liu, J., and Ma, Y., 2016, "A Survey of Manufacturing Oriented Topology Optimization Methods," Advances in Engineering Software, **100**, pp. 161–175.
[67] Sigmund, O., and Maute, K., 2013, "Topology Optimization Approaches," Struct. Multi. Optim., **48**(6), pp. 1031–1055.
[68] Guo, X., Li, W., and Iorio, F., 2016, "Convolutional Neural Networks for Steady Flow Approximation," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, Aug. 13–17, ACM, pp. 481–490.
[69] Oh, S., Jung, Y., Kim, S., Lee, I., and Kang, N., 2019, "Deep Generative Design: Integration of Topology Optimization and Generative Models," ASME J. Mech., Des., **141**(11), p. 111405.
[70] Lei, X., Liu, C., Du, Z., Zhang, W., and Guo, X., 2019, "Machine Learning-Driven Real-Time Topology Optimization Under Moving Morphable Component-Based Framework," ASME J. Appl. Mech., **86**(1), p. 011004.
[71] Montgomery, D. C., 2012, *Design and Analysis of Experiments*, 8th ed, John Wiley & Sons Inc., Hoboken, NJ.
[72] Qi, C. R., Su, H., Mo, K., and Guibas, L. J., 2017, "Pointnet: Deep Learning on Point Sets for 3d Classification and Segmentation," The IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, July 21–26.
[73] Chang, A. X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H. and Xiao, J., 2015, "Shapenet: An Information-Rich 3d Model Repository," eprint arXiv preprint arXiv:1512.03012.
[74] Kondoh, T., Matsumori, T., and Kawamoto, A., 2012, "Drag Minimization and Lift Maximization in Laminar Flows via Topology Optimization Employing Simple Objective Function Expressions Based on Body Force Integration," Struct. Multi. Optim., **45**(5), pp. 693–701.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/142/7/071701/6577896/md_142_7_071701.pdf by Universita Di Pisa user on 24 November 2025